

Pandas Masterclass - Chapter 1

Introduction to Pandas

Pandas is the most widely used Python library for working with tabular data such as CSV, Excel and SQL datasets.

Import Pandas

```
import pandas as pd
```

Library	Purpose
NumPy	Arrays & Numerical Computing
Pandas	Tabular Data Analysis
Matplotlib	Visualization
Scikit-learn	Machine Learning

Create Your First DataFrame

```
import pandas as pd
data={"Name":["Aman", "Riya", "Rohit"], "Marks":[85, 90, 78]}
df=pd.DataFrame(data)
print(df)
```

Check Pandas Version

```
print(pd.__version__)
```

Why Pandas?

Load datasets, clean missing values, filter rows, analyze data and prepare datasets for Machine Learning.

Common Mistakes

1. Forgetting **import pandas as pd**
2. Writing dataframe instead of DataFrame.
3. Confusing Series and DataFrame.

Practice Questions

1. Import pandas.
2. Create a DataFrame with Name and Age.
3. Print the DataFrame.
4. Check pandas version.
5. Write two differences between NumPy and Pandas.

Pandas Masterclass - Chapter 2

Series

A Series is a one-dimensional labeled array in Pandas. It can store integers, floats, strings, and other data types.

1. Creating a Series

```
import pandas as pd
s=pd.Series([10,20,30,40])
print(s)
Output:
0 10
1 20
2 30
3 40
```

2. Custom Index

```
s=pd.Series([85,90,78],index=["Aman", "Riya", "Rohit"])
print(s)
Output:
Aman 85
Riya 90
Rohit 78
```

3. Accessing Data

```
print(s[0])
print(s["Aman"])
Access values using index labels or positions.
```

4. Series from Dictionary

```
data={"Math":90,"Science":85,"English":88}
s=pd.Series(data)
print(s)
```

Operation	Example	Result
Addition	s+5	Adds 5 to every value
Multiplication	s*2	Doubles every value
Maximum	s.max()	Largest value
Minimum	s.min()	Smallest value
Mean	s.mean()	Average
Sum	s.sum()	Total

5. Useful Attributes

```
print(s.index)
print(s.values)
print(s.dtype)
print(s.shape)
```

Machine Learning Use

Series are commonly used to store a single feature (column) such as Age, Salary, or Marks. They are also used as target variables (labels) in supervised learning.

Common Mistakes

- Forgetting that Series has only one dimension.
- Confusing a Series with a DataFrame.
- Mixing integer indexing and label indexing carelessly.

Practice Questions

1. Create a Series of five numbers.
2. Create a Series with custom names as indices.
3. Create a Series from a dictionary.
4. Find mean, max, min and sum.
5. Add 10 to every element of a Series.

Pandas Masterclass - Chapter 3

DataFrame Basics

A DataFrame is a two-dimensional table consisting of rows and columns. It is the most commonly used Pandas data structure.

1. Create DataFrame from Dictionary

```
import pandas as pd
data={ "Name":["Aman", "Riya", "Rohit"],
      "Age": [21, 20, 22],
      "Marks": [85, 92, 78]}
```

```
df=pd.DataFrame(data)
print(df)
```

2. Create DataFrame from List

```
data=[ [1, "Aman", 85],
        [2, "Riya", 92],
        [3, "Rohit", 78]]
```

```
df=pd.DataFrame(data,
columns=["ID", "Name", "Marks"])
```

3. Create DataFrame from NumPy Array

```
import numpy as np
arr=np.array([[10, 20], [30, 40]])
df=pd.DataFrame(arr,
columns=["A", "B"])
```

Property	Purpose	Example
Rows	Horizontal records	df.index
Columns	Vertical fields	df.columns
Shape	Rows, Columns	df.shape
Size	Total values	df.size
Values	Underlying array	df.values

4. Access Columns

```
print(df["Name"])
print(df[["Name", "Marks"]])
```

A single bracket returns one column (Series). Double brackets return a DataFrame.

5. Basic Information

```
print(df.shape)
print(df.columns)
print(df.index)
print(df.values)
```

Machine Learning Use

DataFrames store datasets where each row is a sample and each column is a feature (Age, Salary, Marks, etc.). They are the standard input format before preprocessing and model training.

Common Mistakes

- Using single brackets when multiple columns are needed.
- Forgetting to specify column names while creating DataFrames.
- Confusing Series (1D) with DataFrame (2D).

Practice Questions

1. Create a DataFrame from a dictionary.
2. Create a DataFrame from a list.
3. Create a DataFrame from a NumPy array.
4. Print shape, columns and values.
5. Select one column and then two columns.

Pandas Masterclass - Chapter 4

Reading Files

Code, output and explanation for every concept.

1. `pd.read_csv()`

Code

```
import pandas as pd
df=pd.read_csv('students.csv')
print(df)
```

Output

```
Name Age Marks
Aman 20 85
Riya 21 92
Rohit 22 78
```

Explanation

Reads CSV data into a DataFrame.

2. `head()`

Code

```
print(df.head())
```

Output

First 5 rows

Explanation

Displays the first five rows.

3. `tail()`

Code

```
print(df.tail())
```

Output

Last 5 rows

Explanation

Displays the last five rows.

4. `sample()`

Code

```
print(df.sample(2))
```

Output

Two random rows

Explanation

Returns randomly selected rows.

5. read_excel()

Code

```
df=pd.read_excel('students.xlsx')
```

Output

Excel data loaded

Explanation

Reads Excel files.

6. read_json()

Code

```
df=pd.read_json('students.json')
```

Output

JSON data loaded

Explanation

Reads JSON files.

Common Mistakes

- Wrong file path.
- Missing quotes around filename.
- File not present in working directory.

Machine Learning Tip

Most ML datasets are stored as CSV files, so `pd.read_csv()` is one of the first functions you'll use.

Practice Questions

1. Read a CSV file.
2. Display first 10 rows.
3. Display last 3 rows.
4. Print 5 random rows.
5. Read an Excel file.

Pandas Masterclass - Chapter 5

DataFrame Information

Learn how to inspect a DataFrame using code, outputs and concise explanations.

Create DataFrame

Code

```
import pandas as pd
data={'Name': ['Aman', 'Riya', 'Rohit'], 'Age': [20, 21, 22], 'Marks': [85, 92, 78]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Age Marks
0 Aman 20 85
1 Riya 21 92
2 Rohit 22 78
```

Explanation

Creates a DataFrame with 3 rows and 3 columns.

shape

Code

```
print(df.shape)
```

Output

```
(3, 3)
```

Explanation

Returns (rows, columns).

columns

Code

```
print(df.columns)
```

Output

```
Index(['Name', 'Age', 'Marks'], dtype='object')
```

Explanation

Displays column names.

index

Code

```
print(df.index)
```

Output

```
RangeIndex(start=0, stop=3, step=1)
```

Explanation

Displays row labels.

dtypes

Code

```
print(df.dtypes)
```

Output

```
Name object  
Age int64  
Marks int64
```

Explanation

Shows data type of each column.

info()

Code

```
df.info()
```

Output

```
RangeIndex: 3 entries...  
3 columns  
dtypes: int64(2), object(1)
```

Explanation

Gives a complete summary of the DataFrame.

describe()

Code

```
print(df.describe())
```

Output

```
count mean std min 25% 50% 75% max
```

Explanation

Shows statistical summary for numeric columns.

values

Code

```
print(df.values)
```

Output

```
[['Aman' 20 85] ['Riya' 21 92] ['Rohit' 22 78]]
```

Explanation

Returns the DataFrame as a NumPy array.

size

Code

```
print(df.size)
```

Output

```
9
```

Explanation

Total elements = rows × columns.

Quick Revision

Function	Purpose
shape	Rows & Columns
columns	Column names
index	Row labels
dtypes	Data types
info()	Complete summary
describe()	Statistics
values	NumPy array
size	Total elements

Machine Learning Tip

Always check `df.info()` and `df.describe()` before training a model to understand data types, missing values and statistics.

Practice Questions

1. Create a DataFrame with 5 rows.
2. Print shape.
3. Print columns.
4. Print index.
5. Print dtypes.
6. Use `info()`.
7. Use `describe()`.
8. Print values.
9. Print size.
10. Explain shape vs size.

Pandas Masterclass - Chapter 6

Selecting Data

Learn how to select columns, rows and individual cells from a DataFrame using practical examples.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit'],
      'Age':[20, 21, 22],
      'Marks':[85, 92, 78]}
df=pd.DataFrame(data)
```

Output

```
Name Age Marks
0 Aman 20 85
1 Riya 21 92
2 Rohit 22 78
```

Explanation

Sample DataFrame used throughout this chapter.

1. Select One Column

Code

```
print(df['Name'])
```

Output

```
0 Aman
1 Riya
2 Rohit
```

Explanation

Returns a Series.

2. Select Multiple Columns

Code

```
print(df[['Name', 'Marks']])
```

Output

```
Name Marks
0 Aman 85
1 Riya 92
2 Rohit 78
```

Explanation

Returns a DataFrame.

3. Select First Row

Code

```
print(df.iloc[0])
```

Output

Name Aman
Age 20
Marks 85

Explanation

Returns the first row.

4. Select Multiple Rows

Code

```
print(df.iloc[0:2])
```

Output

Rows 0 and 1

Explanation

Slicing selects rows from start to end-1.

5. Select a Single Cell

Code

```
print(df.at[1, 'Marks'])
```

Output

92

Explanation

Returns one specific value using row label and column name.

6. Select Using iat

Code

```
print(df.iat[2,1])
```

Output

22

Explanation

Returns value using integer row and column positions.

Quick Revision

Method	Returns
df['Name']	One column (Series)
df[['A','B']]	Multiple columns (DataFrame)
iloc[]	Integer position selection
at[]	Single value by label
iat[]	Single value by position

Machine Learning Tip

Feature selection begins by selecting the required columns from a DataFrame. This is one of the first preprocessing steps in ML.

Common Mistakes

- Using single brackets for multiple columns.
- Confusing Series with DataFrame.
- Mixing label-based and integer-based indexing.

Practice Questions

1. Select one column.
2. Select two columns.
3. Print the second row.
4. Print first two rows.
5. Print the Marks of Riya using `at()`.
6. Print Age of Rohit using `iat()`.

Pandas Masterclass - Chapter 7

loc[] and iloc[]

Learn label-based and position-based indexing with practical examples.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit'],
      'Age':[20, 21, 22],
      'Marks':[85, 92, 78]}
df=pd.DataFrame(data, index=['A', 'B', 'C'])
```

Output

```
Name Age Marks
A Aman 20 85
B Riya 21 92
C Rohit 22 78
```

Explanation

Custom index is used to demonstrate loc[].

1. loc[] - Select One Row

Code

```
print(df.loc['B'])
```

Output

```
Name Riya
Age 21
Marks 92
```

Explanation

loc[] selects data using row labels.

2. loc[] - Select Multiple Rows

Code

```
print(df.loc[['A', 'C']])
```

Output

```
Aman and Rohit rows
```

Explanation

Pass a list of labels.

3. loc[] - Row and Column

Code

```
print(df.loc['B', 'Marks'])
```

Output

```
92
```

Explanation

Returns a single value using labels.

4. `iloc[]` - Select One Row

Code

```
print(df.iloc[1])
```

Output

```
Name Riya  
Age 21  
Marks 92
```

Explanation

`iloc[]` uses integer positions.

5. `iloc[]` - Slicing

Code

```
print(df.iloc[0:2,0:2])
```

Output

```
Name Age  
A Aman 20  
B Riya 21
```

Explanation

First two rows and first two columns.

6. `loc[]` vs `iloc[]`

Code

```
df.loc['A']  
df.iloc[0]
```

Output

Both return Aman's row

Explanation

`loc` uses labels while `iloc` uses integer positions.

Quick Comparison

Method	Based On	Example
<code>loc[]</code>	Labels	<code>df.loc['A','Marks']</code>
<code>iloc[]</code>	Positions	<code>df.iloc[0,2]</code>
<code>at[]</code>	Single label value	<code>df.at['A','Age']</code>
<code>iat[]</code>	Single position value	<code>df.iat[0,1]</code>

Machine Learning Tip

`loc[]` is useful when datasets have meaningful row labels. `iloc[]` is commonly used during feature selection by column position.

Common Mistakes

- Using `loc[]` with integer positions when labels are different.
- Confusing `iloc[]` slicing with Python indexing.
- Mixing labels and positions in the same call.

Practice Questions

1. Select row 'C' using `loc[]`.
2. Select second row using `iloc[]`.
3. Print Marks of row 'B' using `loc[]`.
4. Select first two rows and first two columns using `iloc[]`.
5. Compare `loc[]` and `iloc[]` in your own words.

Pandas Masterclass - Chapter 8

Filtering Data

Filtering helps select only the rows that satisfy specific conditions. This is one of the most frequently used operations in data analysis and Machine Learning.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit', 'Simran'],
      'Age':[20, 21, 22, 19],
      'Marks':[85, 92, 78, 65]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Age Marks
0 Aman 20 85
1 Riya 21 92
2 Rohit 22 78
3 Simran 19 65
```

Explanation

This dataset is used in all filtering examples.

1. Filter One Condition

Code

```
print(df[df['Marks']>80])
```

Output

```
Name Age Marks
0 Aman 20 85
1 Riya 21 92
```

Explanation

Returns only students whose Marks are greater than 80.

2. AND Condition (&)

Code

```
print(df[(df['Marks']>70) & (df['Age']>20)])
```

Output

```
Name Age Marks
1 Riya 21 92
2 Rohit 22 78
```

Explanation

Both conditions must be True. Always wrap each condition in parentheses.

3. OR Condition (|)

Code

```
print(df[(df['Marks']>90) | (df['Age']<20)])
```

Output

```
Name Age Marks
1 Riya 21 92
3 Simran 19 65
```

Explanation

At least one condition must be True.

4. isin()

Code

```
print(df[df['Name'].isin(['Aman', 'Rohit'])])
```

Output

```
Name Age Marks
0 Aman 20 85
2 Rohit 22 78
```

Explanation

Checks whether values exist in a given list.

5. between()

Code

```
print(df[df['Marks'].between(70, 90)])
```

Output

```
Name Age Marks
0 Aman 20 85
2 Rohit 22 78
```

Explanation

Returns values between the lower and upper limits (inclusive).

6. not equal (!=)

Code

```
print(df[df['Name']!='Riya'])
```

Output

```
Aman, Rohit and Simran rows
```

Explanation

Filters out rows where Name is Riya.

Quick Revision

Operator	Meaning	Example
==	Equal	df['Age']==20
>	Greater Than	df['Marks']>80
<	Less Than	df['Age']<21
&	AND	(A)&(B)

	OR	(A) (B)
!=	Not Equal	df['Name']!= 'Aman'
isin()	Match from list	df['Name'].isin([...])
between()	Range	df['Marks'].between(70,90)

Machine Learning Tip

Filtering is used to remove invalid records, separate classes, clean datasets, and select data before model training.

Common Mistakes

- Using 'and'/'or' instead of '&'/'|'.
- Forgetting parentheses around conditions.
- Using '=' instead of '==' in comparisons.

Practice Questions

1. Print students with Marks > 75.
2. Print students with Age >= 20 and Marks > 80.
3. Print students whose name is Aman or Simran.
4. Use isin() to select two names.
5. Use between() to filter marks between 60 and 90.

Pandas Masterclass - Chapter 9

Sorting Data

Sorting helps arrange rows based on column values or indexes. It is widely used before analysis and visualization.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit', 'Simran'],
      'Age':[20, 21, 22, 19],
      'Marks':[85, 92, 78, 65]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Age Marks
0 Aman 20 85
1 Riya 21 92
2 Rohit 22 78
3 Simran 19 65
```

Explanation

We'll use this DataFrame for all sorting examples.

1. sort_values() - Ascending

Code

```
print(df.sort_values('Marks'))
```

Output

```
Simran 65
Rohit 78
Aman 85
Riya 92
```

Explanation

Sorts rows by Marks in ascending order.

2. sort_values() - Descending

Code

```
print(df.sort_values('Marks', ascending=False))
```

Output

```
Riya 92
Aman 85
Rohit 78
Simran 65
```

Explanation

Set ascending=False for descending order.

3. Sort by Multiple Columns

Code

```
print(df.sort_values(['Age', 'Marks']))
```

Output

Rows sorted by Age first, then Marks

Explanation

If two rows have the same Age, Marks decides the order.

4. sort_index()

Code

```
print(df.sort_index(ascending=False))
```

Output

Rows displayed with indexes 3,2,1,0

Explanation

Sorts using row indexes instead of column values.

5. inplace=True

Code

```
df.sort_values('Marks', inplace=True)
```

Output

Original DataFrame becomes sorted

Explanation

Changes the existing DataFrame without creating a new one.

6. Reset Index

Code

```
df=df.sort_values('Marks').reset_index(drop=True)  
print(df)
```

Output

Index becomes 0,1,2,3

Explanation

Useful after sorting to get continuous indexing.

Quick Revision

Function	Purpose
sort_values()	Sort by column values
sort_index()	Sort by index
ascending=False	Descending order
inplace=True	Modify original DataFrame
reset_index(drop=True)	Reset row numbers

Machine Learning Tip

Sorting helps inspect highest and lowest values, rank records, and prepare reports before model building.

Common Mistakes

- Forgetting `ascending=False` for descending order.
- Expecting `sort_values()` to modify the original DataFrame without `inplace=True`.
- Forgetting to reset the index after sorting.

Practice Questions

1. Sort Marks in ascending order.
2. Sort Marks in descending order.
3. Sort by Age and Marks.
4. Sort using row index.
5. Reset the index after sorting.

Pandas Masterclass - Chapter 10

Adding & Removing Columns

Learn how to create, modify, insert, rename and delete columns in a DataFrame.

Create DataFrame

Code

```
import pandas as pd
data={'Name': ['Aman', 'Riya', 'Rohit'],
      'Marks': [85, 92, 78]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks
Aman  85
Riya  92
Rohit 78
```

Explanation

Sample DataFrame used in this chapter.

1. Add a New Column

Code

```
df['Age']=[20, 21, 22]
print(df)
```

Output

```
Name Marks Age
Aman  85  20
Riya  92  21
Rohit 78  22
```

Explanation

Creates a new column called Age.

2. Update Existing Column

Code

```
df['Marks']=df['Marks']+5
print(df)
```

Output

```
Marks become 90, 97, 83
```

Explanation

Updates every value in the Marks column.

3. insert()

Code

```
df.insert(1, 'City', ['Delhi', 'Lucknow', 'Kanpur'])
```

```
print(df)
```

Output

```
Name City Marks Age
```

Explanation

Inserts a column at a specific position.

4. rename()

Code

```
df.rename(columns={'Marks':'Score'}, inplace=True)
print(df)
```

Output

```
Column Marks becomes Score
```

Explanation

Renames one or more column names.

5. drop()

Code

```
df.drop('Age', axis=1, inplace=True)
print(df)
```

Output

```
Age column removed
```

Explanation

Deletes a column. Use axis=1 for columns.

6. Reorder Columns

Code

```
df=df[['Name', 'Score', 'City']]
print(df)
```

Output

```
Columns appear in new order
```

Explanation

Select columns in the desired order.

Quick Revision

Function	Purpose
df['New']=...	Add column
insert()	Insert at position
rename()	Rename columns
drop(axis=1)	Delete column
df[['...']]	Reorder columns

Machine Learning Tip

Feature engineering often involves creating new columns, renaming features, and removing unnecessary columns before training models.

Common Mistakes

- Forgetting `axis=1` while dropping columns.
- Using `inplace=True` when you need the original DataFrame unchanged.
- Misspelling column names in `rename()`.

Practice Questions

1. Add a Salary column.
2. Increase all marks by 10.
3. Insert a City column at position 1.
4. Rename Name to Student_Name.
5. Drop the Age column.
6. Reorder columns.

Pandas Masterclass - Chapter 11

Handling Missing Values

Missing values (NaN) are common in real datasets. Pandas provides simple functions to detect, remove, and fill them.

Create DataFrame

Code

```
import pandas as pd
import numpy as np

data={'Name':['Aman', 'Riya', 'Rohit', 'Simran'],
      'Marks':[85,np.nan,78,90],
      'Age':[20,21,np.nan,19]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks Age
Aman  85.0  20.0
Riya  NaN  21.0
Rohit  78.0  NaN
Simran 90.0  19.0
```

Explanation

Dataset containing missing values (NaN).

1. isnull()

Code

```
print(df.isnull())
```

Output

```
True for missing values, False otherwise
```

Explanation

Detects missing values in every cell.

2. notnull()

Code

```
print(df.notnull())
```

Output

```
False for missing values, True otherwise
```

Explanation

Opposite of isnull().

3. Count Missing Values

Code

```
print(df.isnull().sum())
```

Output

Name 0
Marks 1
Age 1

Explanation

Counts missing values column-wise.

4. dropna()

Code

```
print(df.dropna())
```

Output

Only rows without NaN remain

Explanation

Removes rows containing missing values.

5. fillna() with Constant

Code

```
print(df.fillna(0))
```

Output

NaN values become 0

Explanation

Replaces all missing values with a fixed value.

6. fillna() with Mean

Code

```
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Output

Marks missing value replaced with column mean

Explanation

Best for numeric columns in many ML tasks.

7. fillna() with Median

Code

```
df['Age']=df['Age'].fillna(df['Age'].median())
```

Output

Missing Age replaced with median

Explanation

Useful when data contains outliers.

Quick Revision

Function	Purpose
isnull()	Find missing values

<code>notnull()</code>	Find non-missing values
<code>isnull().sum()</code>	Count missing values
<code>dropna()</code>	Remove missing values
<code>fillna(0)</code>	Replace with constant
<code>fillna(mean)</code>	Replace with average
<code>fillna(median)</code>	Replace with median

Machine Learning Tip

Most ML algorithms cannot work with missing values. Always inspect missing data using `isnull().sum()` before training.

Common Mistakes

- Using `dropna()` without realizing rows will be deleted.
- Filling text columns with numeric values.
- Forgetting to assign the result back or use `inplace=True` where applicable.

Practice Questions

1. Find missing values using `isnull()`.
2. Count missing values.
3. Remove rows containing NaN.
4. Replace NaN with 0.
5. Replace missing Marks with the mean.
6. Replace missing Age with the median.

Pandas Masterclass - Chapter 12

Duplicate Data

Duplicate rows are common in real-world datasets. Pandas provides functions to detect and remove them easily.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Aman', 'Rohit', 'Riya'],
      'Marks':[85, 92, 85, 78, 92]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks
0 Aman 85
1 Riya 92
2 Aman 85
3 Rohit 78
4 Riya 92
```

Explanation

Dataset contains duplicate rows.

1. duplicated()

Code

```
print(df.duplicated())
```

Output

```
False
False
True
False
True
```

Explanation

Returns True for duplicate rows.

2. Show Only Duplicate Rows

Code

```
print(df[df.duplicated()])
```

Output

```
Aman 85
Riya 92
```

Explanation

Displays only duplicated records.

3. drop_duplicates()

Code

```
print(df.drop_duplicates())
```

Output

```
Aman 85  
Riya 92  
Rohit 78
```

Explanation

Removes duplicate rows and keeps the first occurrence.

4. keep='last'

Code

```
print(df.drop_duplicates(keep='last'))
```

Output

```
Last duplicate retained
```

Explanation

Keeps the last occurrence instead of the first.

5. keep=False

Code

```
print(df.drop_duplicates(keep=False))
```

Output

```
Rohit 78
```

Explanation

Removes all duplicate entries, leaving only unique rows.

6. Remove Duplicates Based on One Column

Code

```
print(df.drop_duplicates(subset='Name'))
```

Output

```
Aman 85  
Riya 92  
Rohit 78
```

Explanation

Checks duplicates using only the Name column.

Quick Revision

Function	Purpose
<code> duplicated()</code>	Identify duplicate rows
<code> drop_duplicates()</code>	Remove duplicate rows
<code> subset=</code>	Check selected columns only
<code> keep='first'</code>	Keep first occurrence
<code> keep='last'</code>	Keep last occurrence

keep=False	Remove all duplicates
------------	-----------------------

Machine Learning Tip

Duplicate records can bias machine learning models. Always check for duplicates before training using `duplicated()` and `drop_duplicates()`.

Common Mistakes

- Forgetting to assign the result of `drop_duplicates()` back to the DataFrame.
- Using the wrong subset column.
- Not understanding the `keep` parameter.

Practice Questions

1. Find duplicate rows.
2. Display only duplicate records.
3. Remove duplicate rows.
4. Keep the last duplicate only.
5. Remove duplicates using the `Name` column.
6. Remove every duplicated record using `keep=False`.

Pandas Masterclass - Chapter 13

String Functions

The `.str` accessor lets you perform vectorized string operations on an entire column.

Create DataFrame

Code

```
import pandas as pd
data={'Name':[' aman ', 'RIYA', 'rohit', 'Simran Kumar']}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name
0  aman
1  RIYA
2  rohit
3  Simran Kumar
```

Explanation

We'll use this DataFrame for all examples.

1. `str.upper()`

Code

```
print(df['Name'].str.upper())
```

Output

```
AMAN
RIYA
ROHIT
SIMRAN KUMAR
```

Explanation

Converts all text to uppercase.

2. `str.lower()`

Code

```
print(df['Name'].str.lower())
```

Output

```
aman
riya
rohit
simran kumar
```

Explanation

Converts all text to lowercase.

3. `str.title()`

Code

```
print(df['Name'].str.title())
```

Output

```
Aman  
Riya  
Rohit  
Simran Kumar
```

Explanation

Capitalizes the first letter of each word.

4. str.strip()

Code

```
print(df['Name'].str.strip())
```

Output

```
aman  
RIYA  
rohit  
Simran Kumar
```

Explanation

Removes leading and trailing spaces.

5. str.contains()

Code

```
print(df[df['Name'].str.contains('ri', case=False)])
```

Output

```
RIYA  
Simran Kumar
```

Explanation

Filters rows containing the given text.

6. str.replace()

Code

```
print(df['Name'].str.replace('RIYA', 'Riya'))
```

Output

```
aman  
Riya  
rohit  
Simran Kumar
```

Explanation

Replaces matching text.

7. str.len()

Code

```
print(df['Name'].str.len())
```

Output

6
4
5
13

Explanation

Returns the length of each string.

8. str.split()

Code

```
print(df['Name'].str.split())
```

Output

```
['aman']  
['RIYA']  
['rohit']  
['Simran', 'Kumar']
```

Explanation

Splits text into a list using spaces.

Quick Revision

Function	Purpose
str.upper()	Uppercase
str.lower()	Lowercase
str.title()	Title Case
str.strip()	Remove spaces
str.contains()	Search text
str.replace()	Replace text
str.len()	String length
str.split()	Split into list

Machine Learning Tip

String cleaning is essential before NLP and text classification. Convert text to a consistent format, remove spaces, and search or replace unwanted values.

Common Mistakes

- Forgetting to use the `.str` accessor.
- Not handling missing values before string operations.
- Forgetting `case=False` when performing case-insensitive searches.

Practice Questions

1. Convert a column to uppercase.
2. Convert it to lowercase.
3. Remove extra spaces.
4. Find names containing 'an'.
5. Replace 'Aman' with 'Aman Singh'.
6. Find the length of each string.
7. Split full names into first and last names.

Pandas Masterclass - Chapter 14

Mathematical & Statistical Operations

Pandas provides built-in functions to quickly calculate statistics on numeric data.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit', 'Simran'],
      'Marks':[85, 92, 78, 65],
      'Age':[20, 21, 22, 19]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks Age
0 Aman 85 20
1 Riya 92 21
2 Rohit 78 22
3 Simran 65 19
```

Explanation

Sample DataFrame used for all examples.

1. sum()

Code

```
print(df['Marks'].sum())
```

Output

```
320
```

Explanation

Returns the total of all values in the Marks column.

2. mean()

Code

```
print(df['Marks'].mean())
```

Output

```
80.0
```

Explanation

Returns the average value.

3. median()

Code

```
print(df['Marks'].median())
```

Output

```
81.5
```

Explanation

Returns the middle value after sorting.

4. mode()

Code

```
print(df['Marks'].mode())
```

Output

```
0 65 1 78 2 85 3 92
```

Explanation

Returns the most frequent value(s). If all values are unique, every value is returned.

5. max()

Code

```
print(df['Marks'].max())
```

Output

```
92
```

Explanation

Returns the largest value.

6. min()

Code

```
print(df['Marks'].min())
```

Output

```
65
```

Explanation

Returns the smallest value.

7. count()

Code

```
print(df['Marks'].count())
```

Output

```
4
```

Explanation

Counts non-missing values.

8. std()

Code

```
print(df['Marks'].std())
```

Output

```
≈ 11.85
```

Explanation

Calculates the sample standard deviation.

9. var()

Code

```
print(df['Marks'].var())
```

Output

≈ 140.33

Explanation

Calculates the sample variance.

10. Multiple Statistics

Code

```
print(df[['Marks', 'Age']].describe())
```

Output

count, mean, std, min, 25%, 50%, 75%, max

Explanation

Displays a complete statistical summary for numeric columns.

Quick Revision

Function	Purpose
sum()	Total
mean()	Average
median()	Middle value
mode()	Most frequent value
max()	Largest value
min()	Smallest value
count()	Non-null values
std()	Standard deviation
var()	Variance
describe()	Complete statistics

Machine Learning Tip

Always inspect your dataset using statistical functions before training a model. They help identify unusual values, missing data, and the overall distribution.

Common Mistakes

- Confusing mean() and median().
- Assuming mode() always returns one value.
- Forgetting that std() and var() ignore missing values by default.

Practice Questions

1. Find the total marks.
2. Calculate the average age.
3. Find the median marks.
4. Find the maximum and minimum marks.
5. Calculate standard deviation and variance.
6. Display describe() for the DataFrame.

Pandas Masterclass - Chapter 15

GroupBy

groupby() is used to split data into groups and perform calculations on each group. It is one of the most important Pandas operations.

Create DataFrame

Code

```
import pandas as pd
data={ 'Department':['IT', 'HR', 'IT', 'HR', 'Sales'],
       'Employee':['Aman', 'Riya', 'Rohit', 'Simran', 'Karan'],
       'Salary':[50000, 40000, 60000, 45000, 55000]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Department Employee Salary
IT Aman 50000
HR Riya 40000
IT Rohit 60000
HR Simran 45000
Sales Karan 55000
```

Explanation

Sample employee dataset used throughout the chapter.

1. Group by Department

Code

```
print(df.groupby('Department').sum(numeric_only=True))
```

Output

```
HR 85000
IT 110000
Sales 55000
```

Explanation

Groups rows by Department and calculates the total Salary.

2. Mean Salary

Code

```
print(df.groupby('Department')['Salary'].mean())
```

Output

```
HR 42500
IT 55000
Sales 55000
```

Explanation

Calculates the average salary for each department.

3. Count Employees

Code

```
print(df.groupby('Department')['Employee'].count())
```

Output

```
HR 2  
IT 2  
Sales 1
```

Explanation

Counts employees in each department.

4. Maximum Salary

Code

```
print(df.groupby('Department')['Salary'].max())
```

Output

```
HR 45000  
IT 60000  
Sales 55000
```

Explanation

Returns the highest salary in each department.

5. Multiple Aggregations

Code

```
print(df.groupby('Department')['Salary'].agg(['sum', 'mean', 'max', 'min']))
```

Output

sum mean max min for each department

Explanation

agg() performs multiple calculations in a single statement.

6. Group by Multiple Columns

Code

```
print(df.groupby(['Department', 'Employee'])['Salary'].sum())
```

Output

Salary for every Department-Employee combination

Explanation

Groups data using more than one column.

Quick Revision

Function	Purpose
groupby()	Create groups
sum()	Total
mean()	Average
count()	Count values
max()	Maximum

min()	Minimum
agg()	Multiple aggregations

Machine Learning Tip

groupby() is widely used in exploratory data analysis (EDA) to summarize data by category before feature engineering and model building.

Common Mistakes

- Forgetting to select a column before aggregation when needed.
- Applying sum() to text columns unintentionally.
- Forgetting numeric_only=True in some aggregation scenarios.

Practice Questions

1. Find total salary by department.
2. Find average salary by department.
3. Count employees in each department.
4. Find maximum salary in each department.
5. Use agg() to calculate sum, mean, max and min.
6. Group by Department and Employee.

Pandas Masterclass - Chapter 16

Merge, Join & Concat

Combine multiple DataFrames using `merge()`, `join()` and `concat()`. These functions are essential for real-world data analysis.

Create DataFrames

Code

```
import pandas as pd
df1=pd.DataFrame({'ID':[1,2,3], 'Name':['Aman', 'Riya', 'Rohit']})
df2=pd.DataFrame({'ID':[1,2,4], 'Marks':[85, 92, 78]})
```

Output

```
df1 → ID, Name
df2 → ID, Marks
```

Explanation

Two DataFrames with a common column (ID).

1. Inner Merge

Code

```
print(pd.merge(df1, df2, on='ID'))
```

Output

```
ID Name Marks
1 Aman 85
2 Riya 92
```

Explanation

Returns only matching IDs.

2. Left Merge

Code

```
print(pd.merge(df1, df2, on='ID', how='left'))
```

Output

Includes all rows of df1, missing values become NaN.

Explanation

Keeps every row from the left DataFrame.

3. Right Merge

Code

```
print(pd.merge(df1, df2, on='ID', how='right'))
```

Output

Includes all rows of df2.

Explanation

Keeps every row from the right DataFrame.

4. Outer Merge

Code

```
print(pd.merge(df1, df2, on='ID', how='outer'))
```

Output

IDs 1,2,3,4 with NaN where data is missing.

Explanation

Returns every row from both DataFrames.

5. concat()

Code

```
print(pd.concat([df1, df1], ignore_index=True))
```

Output

Rows of df1 appended one after another.

Explanation

Stacks DataFrames vertically by default.

6. join()

Code

```
df1=df1.set_index('ID')
df2=df2.set_index('ID')
print(df1.join(df2))
```

Output

Name and Marks combined using the index.

Explanation

join() combines DataFrames using indexes.

Quick Revision

Function	Purpose
merge()	Combine using common column
how='inner'	Only matching rows
how='left'	Keep left DataFrame
how='right'	Keep right DataFrame
how='outer'	Keep all rows
concat()	Append DataFrames
join()	Combine using index

Machine Learning Tip

Real ML projects often store information across multiple tables. merge() is commonly used to combine customer, transaction and label datasets before model training.

Common Mistakes

- Using the wrong join key.
- Forgetting to specify how=.
- Confusing concat() with merge(). concat() stacks DataFrames, while merge() matches rows using keys.

Practice Questions

1. Perform an inner merge.
2. Perform a left merge.
3. Perform a right merge.
4. Perform an outer merge.
5. Concatenate two DataFrames.
6. Join two DataFrames using their index.

Pandas Masterclass - Chapter 17

Pivot Tables & Crosstab

Pivot tables summarize data efficiently, while `crosstab()` is used to analyze frequency between categories.

Create DataFrame

Code

```
import pandas as pd
data={ 'Department':['IT', 'IT', 'HR', 'HR', 'Sales'],
       'Employee':['Aman', 'Riya', 'Rohit', 'Simran', 'Karan'],
       'Salary':[50000, 60000, 40000, 45000, 55000]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Department Employee Salary
IT Aman 50000
IT Riya 60000
HR Rohit 40000
HR Simran 45000
Sales Karan 55000
```

Explanation

Sample employee dataset.

1. pivot()

Code

```
print(df.pivot(index='Employee', columns='Department', values='Salary'))
```

Output

Employee vs Department salary table with NaN for missing values

Explanation

`pivot()` reshapes data. It requires unique index-column combinations.

2. pivot_table()

Code

```
print(df.pivot_table(index='Department', values='Salary', aggfunc='mean'))
```

Output

```
Department Mean Salary
HR 42500
IT 55000
Sales 55000
```

Explanation

`pivot_table()` supports duplicate values using aggregation.

3. Multiple Aggregations

Code

```
print(df.pivot_table(index='Department', values='Salary', aggfunc=['mean', 'max', 'min']))
```

Output

Mean, Max and Min salary for each department

Explanation

Multiple statistics can be calculated together.

4. Fill Missing Values

Code

```
print(df.pivot_table(index='Employee', columns='Department', values='Salary', fill_value=0))
```

Output

Missing values shown as 0 instead of NaN

Explanation

fill_value replaces missing entries.

5. crosstab()

Code

```
print(pd.crosstab(df['Department'], df['Employee']))
```

Output

Frequency table of departments and employees

Explanation

Counts occurrences between categorical columns.

Quick Revision

Function	Purpose
pivot()	Reshape data
pivot_table()	Summarize with aggregation
aggfunc	Choose aggregation
fill_value	Replace NaN
crosstab()	Frequency table

Machine Learning Tip

Pivot tables are widely used in Exploratory Data Analysis (EDA) to summarize features before building machine learning models.

Common Mistakes

- Using pivot() when duplicate combinations exist (use pivot_table() instead).
- Forgetting aggfunc in pivot_table().
- Confusing crosstab() with pivot tables.

Practice Questions

1. Create a pivot table of average salary by department.
2. Find maximum salary using `pivot_table()`.
3. Replace missing values using `fill_value`.
4. Create a crosstab of Department and Employee.
5. Explain `pivot()` vs `pivot_table()`.

Pandas Masterclass - Chapter 18

apply(), map() & applymap()

These functions transform data efficiently and are widely used for feature engineering and preprocessing.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit'],
      'Marks':[85, 92, 78],
      'Age':[20, 21, 22]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks Age
0 Aman 85 20
1 Riya 92 21
2 Rohit 78 22
```

Explanation

Dataset used in all examples.

1. Series.map()

Code

```
grade={85:'B', 92:'A', 78:'C'}
print(df['Marks'].map(grade))
```

Output

```
0 B
1 A
2 C
```

Explanation

map() replaces each value using a dictionary or function.

2. Series.apply()

Code

```
print(df['Marks'].apply(lambda x:x+5))
```

Output

```
0 90
1 97
2 83
```

Explanation

apply() applies a custom function to every value in a Series.

3. DataFrame.apply()

Code

```
print(df[['Marks', 'Age']].apply(sum))
```

Output

```
Marks 255  
Age 63
```

Explanation

Applies a function column-wise by default.

4. DataFrame.apply(axis=1)

Code

```
print(df[['Marks', 'Age']].apply(sum, axis=1))
```

Output

```
0 105  
1 113  
2 100
```

Explanation

axis=1 performs the operation row-wise.

5. applymap()

Code

```
print(df[['Marks', 'Age']].applymap(lambda x:x*2))
```

Output

```
Marks Age  
170 40  
184 42  
156 44
```

Explanation

applymap() applies a function to every element of a DataFrame.

6. Create New Column

Code

```
df['Pass']=df['Marks'].apply(lambda x:'Yes' if x>=80 else 'No')  
print(df)
```

Output

```
Name Marks Age Pass  
Aman 85 20 Yes  
Riya 92 21 Yes  
Rohit 78 22 No
```

Explanation

Useful for feature engineering.

Quick Revision

Function	Works On	Purpose
map()	Series	Mapping values

<code>apply()</code>	Series/DataFrame	Custom functions
<code>apply(axis=1)</code>	Rows	Row-wise operation
<code>applymap()</code>	DataFrame	Element-wise transformation

Machine Learning Tip

These functions are frequently used to create new features, encode categories, scale values and transform datasets before training ML models.

Common Mistakes

- Using `map()` on a DataFrame instead of a Series.
- Forgetting `axis=1` for row-wise operations.
- Confusing `apply()` with `applymap()`.

Practice Questions

1. Add 10 to all marks using `apply()`.
2. Map grades using a dictionary.
3. Find row-wise sums using `apply(axis=1)`.
4. Double all numeric values using `applymap()`.
5. Create a Pass/Fail column using `lambda`.

Pandas Masterclass - Chapter 19

Date & Time Handling

Date and time preprocessing is common in machine learning datasets such as sales, finance, weather and attendance.

Create DataFrame

Code

```
import pandas as pd
df=pd.DataFrame({'Date':['2026-06-29', '2026-07-01', '2026-08-15']})
print(df)
```

Output

```
Date
2026-06-29
2026-07-01
2026-08-15
```

Explanation

Sample dataset containing dates stored as strings.

1. Convert to Datetime

Code

```
df['Date']=pd.to_datetime(df['Date'])
print(df.dtypes)
```

Output

```
Date  datetime64[ns]
```

Explanation

Converts string values into datetime format.

2. Extract Year

Code

```
print(df['Date'].dt.year)
```

Output

```
2026
2026
2026
```

Explanation

Returns the year from each date.

3. Extract Month & Day

Code

```
print(df['Date'].dt.month)
print(df['Date'].dt.day)
```

Output

```
6 7 8
```

29 1 15

Explanation

Extracts month and day separately.

4. Day Name & Month Name

Code

```
print(df['Date'].dt.day_name())  
print(df['Date'].dt.month_name())
```

Output

```
Monday ...  
June July August
```

Explanation

Returns readable names.

5. Date Arithmetic

Code

```
df['Next_Week']=df['Date']+pd.Timedelta(days=7)  
print(df)
```

Output

```
Next_Week contains dates 7 days later
```

Explanation

Adds or subtracts time using Timedelta.

6. Difference Between Dates

Code

```
print(df['Next_Week']-df['Date'])
```

Output

```
7 days
```

Explanation

Calculates time differences.

7. Current Date & Time

Code

```
print(pd.Timestamp.now())
```

Output

```
2026-... ..:..:..
```

Explanation

Returns the current system date and time.

Quick Revision

Function	Purpose
----------	---------

<code>pd.to_datetime()</code>	Convert to datetime
<code>dt.year</code>	Extract year
<code>dt.month</code>	Extract month
<code>dt.day</code>	Extract day
<code>dt.day_name()</code>	Weekday name
<code>dt.month_name()</code>	Month name
<code>Timedelta</code>	Date arithmetic
<code>Timestamp.now()</code>	Current date/time

Machine Learning Tip

Date columns are often converted into features like Year, Month, Day or Weekday before training a machine learning model.

Common Mistakes

- Forgetting `pd.to_datetime()` before using `.dt`.
- Applying `.dt` to string columns.
- Ignoring date formats while importing data.

Practice Questions

1. Convert a string column to datetime.
2. Extract year, month and day.
3. Display weekday names.
4. Add 30 days to every date.
5. Find the difference between two dates.

Pandas Masterclass - Chapter 20

File Handling

Learn how to save and load DataFrames in different file formats used in data analysis and machine learning.

Create DataFrame

Code

```
import pandas as pd
data={'Name':['Aman', 'Riya', 'Rohit'],
      'Marks':[85, 92, 78]}
df=pd.DataFrame(data)
print(df)
```

Output

```
Name Marks
0 Aman 85
1 Riya 92
2 Rohit 78
```

Explanation

Sample DataFrame used for saving and loading files.

1. Save CSV

Code

```
df.to_csv('students.csv')
```

Output

```
students.csv created
```

Explanation

Saves the DataFrame as a CSV file.

2. Save CSV without Index

Code

```
df.to_csv('students.csv', index=False)
```

Output

```
CSV file without index column
```

Explanation

index=False prevents row numbers from being saved.

3. Read CSV

Code

```
df=pd.read_csv('students.csv')
print(df.head())
```

Output

```
Name Marks
Aman 85
```

Riya 92
Rohit 78

Explanation

Reads a CSV file into a DataFrame.

4. Save Excel

Code

```
df.to_excel('students.xlsx', index=False)
```

Output

students.xlsx created

Explanation

Saves the DataFrame as an Excel workbook.

5. Read Excel

Code

```
df=pd.read_excel('students.xlsx')
```

Output

Excel data loaded

Explanation

Reads an Excel file into a DataFrame.

6. Save JSON

Code

```
df.to_json('students.json', orient='records')
```

Output

students.json created

Explanation

Stores data in JSON format.

7. Read JSON

Code

```
df=pd.read_json('students.json')
```

Output

JSON data loaded

Explanation

Reads JSON data into a DataFrame.

Quick Revision

Function	Purpose
to_csv()	Save CSV

read_csv()	Read CSV
to_excel()	Save Excel
read_excel()	Read Excel
to_json()	Save JSON
read_json()	Read JSON
index=False	Do not save row index

Machine Learning Tip

Most ML datasets are stored as CSV files. After cleaning data, it is common to save the processed dataset using `to_csv(index=False)` before model training.

Common Mistakes

- Forgetting `index=False` when saving.
- Using an incorrect file path.
- Trying to read a file that does not exist.
- Missing the `openpyxl` package when working with Excel files.

Practice Questions

1. Save a DataFrame as CSV.
2. Save a CSV without the index.
3. Read a CSV file.
4. Save and read an Excel file.
5. Save and read a JSON file.

Pandas Masterclass - Chapter 21

Data Cleaning

Data cleaning prepares raw data for analysis and machine learning by fixing errors, missing values, duplicates and inconsistent formats.

Create DataFrame

Code

```
import pandas as pd import numpy as np data={ 'Name':[' Aman', 'Riya', 'Riya', 'Rohit'], 'Age':['20', '21', '21', np.nan], 'Marks':[85, -1, 92, 78] } df=pd.DataFrame(data) print(df)
```

Output

```
Name Age Marks Aman 20 85 Riya 21 -1 Riya 21 92 Rohit NaN 78
```

Explanation

Sample dataset containing spaces, duplicates, missing values and invalid marks.

1. Remove Extra Spaces

Code

```
df['Name']=df['Name'].str.strip()
```

Output

```
' Aman ' becomes 'Aman'
```

Explanation

strip() removes leading and trailing spaces.

2. Replace Invalid Values

Code

```
df['Marks']=df['Marks'].replace(-1, np.nan)
```

Output

```
-1 becomes NaN
```

Explanation

replace() substitutes unwanted values.

3. Fill Missing Values

Code

```
df['Age']=df['Age'].fillna(df['Age'].mode()[0])
```

Output

```
Missing Age filled with 21
```

Explanation

fillna() replaces missing values.

4. Convert Data Type

Code

```
df['Age']=df['Age'].astype(int)
```

Output

Age column becomes int

Explanation

astype() changes the data type.

5. Remove Duplicates

Code

```
df=df.drop_duplicates()
```

Output

Duplicate Riya row removed

Explanation

Keeps unique records only.

6. Simple Outlier Filter

Code

```
clean=df[df['Marks']<=100]
```

Output

Rows with Marks >100 removed

Explanation

Boolean filtering removes obvious outliers.

7. Complete Cleaning Pipeline

Code

```
df['Name']=df['Name'].str.strip()  
df['Marks']=df['Marks'].replace(-1,np.nan)  
df=df.fillna({'Marks':df['Marks'].mean()}) df=df.drop_duplicates()
```

Output

Clean DataFrame ready for analysis

Explanation

Combines multiple cleaning steps.

Quick Revision

Function	Purpose
str.strip()	Remove spaces
replace()	Replace values
fillna()	Handle missing values
astype()	Convert data type
drop_duplicates()	Remove duplicates
Boolean Filtering	Remove outliers

Machine Learning Tip

Data cleaning usually consumes most of the time in a machine learning project. A clean dataset leads to more reliable models and better predictions.

Common Mistakes

- Forgetting to assign cleaned data back to the DataFrame.
- Converting data types before handling missing values.
- Removing valid records while filtering outliers.

Practice Questions

1. Remove leading/trailing spaces.
2. Replace -1 with NaN.
3. Fill missing values with mode or mean.
4. Convert Age to integer.
5. Remove duplicate rows.
6. Filter out marks greater than 100.
7. Perform a complete cleaning pipeline.

Pandas Masterclass - Chapter 22

Data Visualization with Pandas

Pandas provides built-in plotting methods powered by Matplotlib for quick data visualization.

Create DataFrame

Code

```
import pandas as pd import matplotlib.pyplot as plt
data={'Month':['Jan', 'Feb', 'Mar', 'Apr'], 'Sales':[120,150,180,160]}
df=pd.DataFrame(data) print(df)
```

Output

```
Month Sales 0 Jan 120 1 Feb 150 2 Mar 180 3 Apr 160
```

Explanation

Sample sales dataset used for all charts.

1. Line Chart

Code

```
df.plot(x='Month',y='Sales',kind='line') plt.show()
```

Output

A line chart showing sales trend from Jan to Apr.

Explanation

Best for showing trends over time.

2. Bar Chart

Code

```
df.plot(x='Month',y='Sales',kind='bar') plt.show()
```

Output

A bar chart with one bar per month.

Explanation

Compares values across categories.

3. Histogram

Code

```
df['Sales'].plot(kind='hist') plt.show()
```

Output

Histogram of Sales values.

Explanation

Shows the distribution of numeric data.

4. Box Plot

Code

```
df['Sales'].plot(kind='box') plt.show()
```

Output

Box plot displaying spread and possible outliers.

Explanation

Useful for detecting outliers.

5. Pie Chart

Code

```
df.set_index('Month')['Sales'].plot(kind='pie', autopct='%1.1f%%')  
plt.show()
```

Output

Pie chart showing percentage contribution of each month.

Explanation

Displays proportions of a whole.

6. Scatter Plot

Code

```
df.plot(x='Month', y='Sales', kind='scatter')
```

Output

Raises an error because x is non-numeric.

Explanation

Scatter plots require numeric x and y values.

7. Scatter Plot (Correct)

Code

```
df['Month_No']=[1,2,3,4] df.plot(x='Month_No', y='Sales', kind='scatter')  
plt.show()
```

Output

Scatter plot of Sales vs Month number.

Explanation

Shows the relationship between two numeric variables.

Quick Revision

Chart	Purpose
Line	Trend over time
Bar	Category comparison
Histogram	Distribution
Box	Outlier detection
Pie	Percentage share
Scatter	Relationship between variables

Machine Learning Tip

Visualization helps identify trends, skewed distributions, and outliers before training machine learning models.

Common Mistakes

- Forgetting `plt.show()`.
- Using scatter plots with non-numeric columns.
- Choosing the wrong chart type for the data.

Practice Questions

1. Draw a line chart.
2. Create a bar chart.
3. Plot a histogram.
4. Create a box plot.
5. Draw a pie chart.
6. Create a scatter plot using two numeric columns.

Pandas Masterclass - Chapter 23

Real Machine Learning Dataset Workflow

This chapter combines Pandas concepts into a complete preprocessing pipeline before training a machine learning model.

1. Load Dataset

Code

```
import pandas as pd df=pd.read_csv('students.csv') print(df.head())
```

Output

```
Name Age Marks Aman 20 85 Riya 21 NaN Rohit 22 78
```

Explanation

Load a CSV dataset into a DataFrame.

2. Explore Dataset

Code

```
print(df.info()) print(df.describe())
```

Output

Displays rows, columns, data types and statistics.

Explanation

Always inspect data before cleaning.

3. Handle Missing Values

Code

```
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Output

NaN replaced with average marks.

Explanation

Fill missing numeric values with the mean.

4. Remove Duplicates

Code

```
df=df.drop_duplicates()
```

Output

Duplicate rows removed.

Explanation

Keep only unique records.

5. Feature Selection

Code

```
X=df[['Age', 'Marks']] y=df['Result']
```


Output

X contains features y contains target

Explanation

Separate input features and target labels.

6. Filter Records

Code

```
df=df[df['Marks']>=40]
```

Output

Only passed students remain.

Explanation

Filter unnecessary rows before training.

7. Save Clean Dataset

Code

```
df.to_csv('students_clean.csv',index=False)
```

Output

students_clean.csv created

Explanation

Save the processed dataset for reuse.

Complete ML Workflow

Step	Purpose
Load	Read dataset
Explore	Understand data
Clean	Fix missing values & duplicates
Select Features	Choose ML inputs
Filter	Keep useful records
Save	Store cleaned dataset

Machine Learning Pipeline

Read CSV → Inspect → Clean Missing Values → Remove Duplicates → Select Features → Filter Records → Save Clean Dataset → Train Model

Common Mistakes

- Training on raw data without cleaning.
- Forgetting to separate features (X) and target (y).
- Saving CSV with unwanted index column.

Practice Questions

1. Load a CSV dataset.
2. Display `info()` and `describe()`.
3. Fill missing values.
4. Remove duplicates.
5. Select features and target.
6. Filter rows with $\text{Marks} \geq 50$.
7. Save the cleaned dataset.

Pandas Masterclass - Chapter 24

Pandas Interview Questions

Frequently asked Pandas interview questions with concise answers for internships and placements.

1. What is Pandas?

A Python library for data manipulation and analysis.

2. Difference between Series and DataFrame?

Series is 1D; DataFrame is 2D.

3. read_csv()

Reads CSV files into a DataFrame.

4. head() vs tail()

First rows vs last rows.

5. loc[] vs iloc[]

loc uses labels; iloc uses integer positions.

6. at[] vs iat[]

Fast access to a single value by label vs position.

7. shape

Returns (rows, columns).

8. info()

Displays data types, null values and memory usage.

9. describe()

Statistical summary of numeric columns.

10. isnull()

Detects missing values.

11. fillna()

Replaces missing values.

12. dropna()

Removes rows/columns with missing values.

13. duplicated()

Finds duplicate rows.

14. drop_duplicates()

Removes duplicate rows.

15. sort_values()

Sorts by column values.

16. groupby()

Groups data for aggregation.

17. merge()

Combines DataFrames using common keys.

18. concat()

Stacks DataFrames vertically or horizontally.

19. pivot_table()

Creates summarized tables with aggregation.

20. apply()

Applies custom functions to Series/DataFrames.

21. map()

Maps values in a Series.

22. to_datetime()

Converts strings to datetime.

23. to_csv(index=False)

Saves CSV without row index.

24. Common ML workflow?

Load → Explore → Clean → Transform → Save → Train model.

25. Most important Pandas functions?

read_csv, info, describe, loc, iloc, groupby, merge, fillna, drop_duplicates.

Quick Interview Tips

Interview Tip	Advice
Data Cleaning	Explain fillna(), dropna(), duplicates
Indexing	Know loc, iloc, at, iat
Aggregation	Practice groupby and pivot_table
Files	Know read_csv and to_csv
Projects	Mention using Pandas in ML preprocessing

Pandas Masterclass

Practice Question Answers (Chapters 1–24)

This workbook contains concise answers and sample solutions for the practice questions from all chapters.

Chapter 1

1. Create a Series: `pd.Series([10,20,30])`
2. Dictionary to Series: `pd.Series({'A':1,'B':2})`
3. Access first element: `s[0]`
4. Find datatype: `s.dtype`
5. Convert list to Series: `pd.Series(my_list)`

Chapter 2

1. Create DataFrame from dictionary: `pd.DataFrame(data)`
2. Create DataFrame from list: `pd.DataFrame(list_data)`
3. Create from NumPy array: `pd.DataFrame(arr)`
4. Display shape: `df.shape`
5. Select columns: `df[['Name','Marks']]`

Chapter 3

1. Create DataFrame using dictionary/list/NumPy.
2. Use `df.columns`, `df.index`, `df.values`.
3. Display `df.shape` and `df.size`.
4. Access one and multiple columns.

Chapter 4

1. Read CSV: `pd.read_csv('file.csv')`
2. Read Excel: `pd.read_excel('file.xlsx')`
3. Read JSON: `pd.read_json('file.json')`
4. First rows: `df.head()`
5. Last rows: `df.tail()`

Chapter 5

1. Use `df.info()`, `df.describe()`, `df.shape`, `df.columns`, `df.index`, `df.values`.

Chapter 6

1. One column: `df['Name']`
2. Multiple columns: `df[['Name','Marks']]`
3. Row: `df.iloc[1]`

4. Cell: `df.at[1,'Marks']`

Chapter 7

1. loc by label: `df.loc['A']`
2. iloc by position: `df.iloc[0]`
3. Slice: `df.iloc[0:2,0:2]`

Chapter 8

1. Marks>75: `df[df['Marks']>75]`
2. AND: `(A)&(B)`
3. OR: `(A)|(B)`
4. `isin()`: `df['Name'].isin([...])`
5. `between()`: `df['Marks'].between(60,90)`

Chapter 9

1. Sort ascending: `df.sort_values('Marks')`
2. Descending: `ascending=False`
3. Reset index: `reset_index(drop=True)`

Chapter 10

1. Add: `df['Age']=...`
2. Rename: `df.rename(...)`
3. Drop: `df.drop('Age',axis=1)`
4. Insert: `df.insert(...)`

Chapter 11

1. Find NaN: `df.isnull()`
2. Count: `df.isnull().sum()`
3. Fill: `fillna(mean)`
4. Drop: `dropna()`

Chapter 12

1. Find duplicates: `duplicated()`
2. Remove: `drop_duplicates()`
3. Subset: `drop_duplicates(subset='Name')`

Chapter 13

1. Upper: `str.upper()`
2. Lower: `str.lower()`
3. Replace: `str.replace()`

4. Contains: `str.contains()`

Chapter 14

1. Sum: `sum()`
2. Mean: `mean()`
3. Median: `median()`
4. Describe: `describe()`

Chapter 15

1. Group sum: `groupby().sum()`
2. Mean: `groupby()['Salary'].mean()`
3. Multiple: `agg(['sum','mean'])`

Chapter 16

1. Inner merge: `merge(...,how='inner')`
2. Left/Right/Outer merge
3. Concat: `pd.concat([...])`
4. Join: `df1.join(df2)`

Chapter 17

1. Pivot: `pivot()`
2. Pivot table: `pivot_table()`
3. Crosstab: `pd.crosstab()`

Chapter 18

1. `map()`: `map(dict)`
2. `apply()`: `apply(lambda x:x+5)`
3. `applymap()`: `applymap(lambda x:x*2)`

Chapter 19

1. Convert: `pd.to_datetime()`
2. Year: `dt.year`
3. Add days: `+pd.Timedelta(days=30)`

Chapter 20

1. Save CSV: `to_csv(index=False)`
2. Read CSV: `read_csv()`
3. Save/Read Excel & JSON

Chapter 21

1. `strip()`, `replace()`, `fillna()`, `astype()`, `drop_duplicates()`, filtering

Chapter 22

1. Line: `plot(kind='line')`
2. Bar: `kind='bar'`
3. Histogram: `kind='hist'`
4. Pie: `kind='pie'`

Chapter 23

1. Workflow: Read → Explore → Clean → Feature Selection → Save

Chapter 24

1. Key interview topics: `loc/iloc`, `groupby`, `merge`, `fillna`, `describe`, `info`, `apply`.

Machine Learning Preprocessing

Chapter 1 - Encoding (Detailed Notes)

Encoding converts categorical (text) values into numbers so Machine Learning algorithms can understand them.

1. Why Encoding?

Code / Example

```
Gender = ['Male', 'Female', 'Male']
```

Output

ML cannot directly use text values.

Explanation

Algorithms work with numbers, so categorical values must be converted into numeric form.

2. Types of Encoding

Code / Example

Label Encoding One-Hot Encoding Ordinal Encoding

Explanation

These are the three most common encoding techniques used in Machine Learning.

3. Label Encoding

Code / Example

```
from sklearn.preprocessing import LabelEncoder import pandas as pd
df=pd.DataFrame({'Gender':['Male','Female','Male']}) le=LabelEncoder()
df['Gender']=le.fit_transform(df['Gender']) print(df)
```

Output

Gender 0 1 1 0 2 1

Explanation

Each category receives an integer. Suitable for binary labels or target columns.

Example where Label Encoding is Wrong

Code / Example

City Delhi Lucknow Kanpur

Output

Delhi=0 Kanpur=1 Lucknow=2

Explanation

Cities have no natural order. The model may incorrectly assume $2 > 1 > 0$.

4. One-Hot Encoding using Pandas

Code / Example

```
import pandas as pd df=pd.DataFrame({'Gender':['Male','Female','Male']})
print(pd.get_dummies(df, dtype=int))
```

Output

Gender_Female Gender_Male 0 0 1 1 1 0 2 0 1

Explanation

Creates one binary column for each category. Best for nominal (unordered) data.

One-Hot Encoding using Scikit-learn

Code / Example

```
from sklearn.preprocessing import OneHotEncoder
enc=OneHotEncoder(sparse_output=False) enc.fit_transform(df[['Gender']])
```

Output

```
[[0. 1.] [1. 0.] [0. 1.]]
```

Explanation

Preferred when building Scikit-learn pipelines.

5. Ordinal Encoding

Code / Example

```
from sklearn.preprocessing import OrdinalEncoder
df=pd.DataFrame({'Size':['Small','Medium','Large']})
enc=OrdinalEncoder(categories=[['Small','Medium','Large']])
print(enc.fit_transform(df))
```

Output

```
[[0.] [1.] [2.]]
```

Explanation

Use only when categories have a meaningful order.

6. Comparison Table

Encoding	Use	Example
Label	Binary/Target	Yes/No
One-Hot	Unordered Categories	City, Gender, Color
Ordinal	Ordered Categories	Small<Medium<Large

7. Common Mistakes

- Don't use Label Encoding for unordered categories like City.
- Convert training data and test data consistently.
- Save the fitted encoder for future predictions.
- Prefer One-Hot Encoding for most categorical input features.

8. Interview & Practice

Interview Questions

1. Why is encoding required?
2. Label Encoding vs One-Hot Encoding?
3. When should Ordinal Encoding be used?
4. Why is Label Encoding not suitable for City?

Practice Questions

1. Encode Gender using LabelEncoder.
2. Encode City using `pd.get_dummies()`.
3. Encode Size using OrdinalEncoder.
4. Compare all three encoding methods with examples.

Machine Learning Preprocessing

Chapter 2 - Missing Value Handling

Missing values (NaN) are common in real datasets. Before training a model, they must be detected and handled correctly.

1. Why Missing Values?

Code

```
Age = [20, NaN, 22] Marks = [85, 90, NaN]
```

Output

NaN means data is missing.

Explanation

Missing values may occur due to survey errors, unavailable information or faulty sensors.

2. Detect Missing Values

Code

```
import pandas as pd import numpy as np
df=pd.DataFrame({'Age':[20,np.nan,22], 'Marks':[85,90,np.nan]})
print(df.isnull())
```

Output

```
Age Marks 0 False False 1 True False 2 False True
```

Explanation

isnull() returns True where data is missing.

3. Count Missing Values

Code

```
print(df.isnull().sum())
```

Output

```
Age 1 Marks 1
```

Explanation

Counts missing values in each column.

4. Remove Missing Values

Code

```
print(df.dropna())
```

Output

Only rows without NaN remain.

Explanation

dropna() removes rows containing missing values.

5. Fill with Mean

Code

```
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Output

NaN replaced by average marks.

Explanation

Best for normally distributed numeric data.

6. Fill with Median

Code

```
df['Age']=df['Age'].fillna(df['Age'].median())
```

Output

NaN replaced by median.

Explanation

Preferred when outliers exist.

7. Fill with Mode

Code

```
df['City']=df['City'].fillna(df['City'].mode()[0])
```

Output

Missing city replaced with most frequent city.

Explanation

Useful for categorical columns.

8. Forward & Backward Fill

Code

```
df.fffll() df.bfill()
```

Output

Previous value / Next value used.

Explanation

Useful in time-series datasets.

9. Comparison Table

Method	Best Used For
dropna()	Remove incomplete rows
fillna(mean)	Numeric data
fillna(median)	Numeric with outliers
fillna(mode)	Categorical data
ffill()	Time-series
bfill()	Time-series

10. Common Mistakes

- Dropping too many rows with `dropna()`.
- Filling categorical data with mean.
- Ignoring missing values before model training.

11. Interview Questions

1. What is NaN?
2. Mean vs Median imputation?
3. When should you use mode?
4. What is forward fill?
5. Why are missing values harmful for ML?

12. Practice Questions

1. Find missing values using `isnull()`.
2. Count missing values.
3. Fill Marks with mean.
4. Fill Age with median.
5. Fill City with mode.
6. Remove rows with NaN.
7. Use forward fill and backward fill.

Machine Learning Preprocessing

Chapter 3 - Feature Scaling

Feature scaling brings features to a similar range so that ML algorithms treat every feature fairly.

1. Why Feature Scaling?

Code

```
Age=[18,20,22] Salary=[20000,50000,90000]
```

Output

Salary values are much larger than Age values.

Explanation

Algorithms may give more importance to Salary because of its larger scale.

2. Normalization (MinMaxScaler)

Code

```
from sklearn.preprocessing import MinMaxScaler scaler=MinMaxScaler()  
scaled=scaler.fit_transform(df[['Age', 'Salary']])
```

Output

Values become between 0 and 1

Explanation

Formula: $(x - \min) / (\max - \min)$. Best when data has known minimum and maximum.

3. Standardization (StandardScaler)

Code

```
from sklearn.preprocessing import StandardScaler scaler=StandardScaler()  
scaled=scaler.fit_transform(df[['Age', 'Salary']])
```

Output

Mean $\approx$ 0 Standard Deviation $\approx$ 1

Explanation

Formula: $(x - \text{mean}) / \text{std}$. Most commonly used scaling technique.

4. RobustScaler

Code

```
from sklearn.preprocessing import RobustScaler  
scaled=RobustScaler().fit_transform(df[['Age', 'Salary']])
```

Output

Median becomes the center of scaling.

Explanation

Uses median and IQR, so it works well when outliers are present.

5. MaxAbsScaler

Code

```
from sklearn.preprocessing import MaxAbsScaler  
scaled=MaxAbsScaler().fit_transform(df[['Age', 'Salary']])
```

Output

Values lie between -1 and 1.

Explanation

Useful for sparse datasets because it preserves zero values.

6. Which Algorithms Need Scaling?

Code

KNN, SVM, K-Means, Logistic Regression, Neural Networks

Output

Scaling Recommended

Explanation

Tree-based models (Decision Tree, Random Forest, XGBoost) generally do not require feature scaling.

7. Comparison Table

Scaler	Range / Property	Best Use
MinMaxScaler	0 to 1	Neural Networks, image data
StandardScaler	Mean=0, Std=1	General-purpose ML
RobustScaler	Median & IQR	Data with outliers
MaxAbsScaler	-1 to 1	Sparse data

8. Common Mistakes

- Scaling the entire dataset before train-test split (causes data leakage).
- Using MinMaxScaler on data with many outliers.
- Fitting the scaler separately on test data instead of using the scaler fitted on training data.

9. Interview Questions

1. Why is feature scaling required?
2. MinMaxScaler vs StandardScaler?
3. Which scaler is best for outliers?
4. Which ML algorithms require scaling?
5. Do Decision Trees need feature scaling?

10. Practice Questions

1. Apply MinMaxScaler to Age and Salary.
2. Standardize a dataset using StandardScaler.
3. Scale data containing outliers with RobustScaler.
4. Apply MaxAbsScaler to sparse data.
5. Explain which scaler you would choose for each scenario.

Machine Learning Preprocessing

Chapter 4 - Train-Test Split

Train-test split divides the dataset into training and testing sets so we can evaluate how well a machine learning model performs on unseen data.

1. Why Train-Test Split?

Code

Dataset → Train Set + Test Set

Output

Typically 80% Train, 20% Test

Explanation

Training data is used to learn patterns, while test data checks model performance on unseen samples.

2. Basic train_test_split()

Code

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=
train_test_split(X,y,test_size=0.2,random_state=42)
```

Output

80% Training Data 20% Testing Data

Explanation

Splits features (X) and target (y) into training and testing sets.

3. test_size Parameter

Code

```
train_test_split(X,y,test_size=0.3)
```

Output

70% Train 30% Test

Explanation

Controls how much data is reserved for testing.

4. random_state

Code

```
train_test_split(X,y,random_state=42)
```

Output

Same split every time

Explanation

Using the same random_state makes experiments reproducible.

5. shuffle Parameter

Code

```
train_test_split(X,y, shuffle=False)
```

Output

Original order is preserved

Explanation

Useful for time-series data where order matters.

6. Stratified Split

Code

```
train_test_split( X,y, test_size=0.2, stratify=y, random_state=42)
```

Output

Class ratio remains the same

Explanation

Recommended for classification datasets with imbalanced classes.

7. Parameter Summary

Parameter	Purpose
test_size	Size of test set
train_size	Size of training set
random_state	Reproducible split
shuffle	Shuffle before splitting
stratify	Maintain class distribution

8. Common Mistakes

- Training and testing on the same data.
- Forgetting random_state, making results change every run.
- Scaling or encoding the entire dataset before splitting (data leakage).
- Not using stratify for imbalanced classification datasets.

9. Interview Questions

1. Why do we split a dataset?
2. What does test_size do?
3. Why is random_state important?
4. What is stratify?
5. When should shuffle=False be used?

10. Practice Questions

1. Split a dataset into 80% training and 20% testing.
2. Use random_state=42.
3. Perform a 70-30 split.
4. Split without shuffling.
5. Perform a stratified split for a classification dataset.

Machine Learning Preprocessing

Chapter 5 - Feature Engineering

Feature Engineering is the process of creating new, more useful features from existing data to improve machine learning model performance.

1. Why Feature Engineering?

Code

```
Age, Joining_Date, Salary
```

Output

```
Create Age_Group, Experience, Salary_Per_Year
```

Explanation

Good features often improve accuracy more than changing the ML algorithm.

2. Create a New Feature

Code

```
df['Total']=df['Maths']+df['Science']  
print(df[['Maths', 'Science', 'Total']])
```

Output

```
Maths Science Total 80 70 150
```

Explanation

New features can combine information from multiple columns.

3. Date Features

Code

```
df['Date']=pd.to_datetime(df['Date']) df['Year']=df['Date'].dt.year  
df['Month']=df['Date'].dt.month
```

Output

```
Date Year Month 2025-06-10 2025 6
```

Explanation

Extracting year, month or weekday is common in sales and finance datasets.

4. Text Features

Code

```
df['Length']=df['Name'].str.len()
```

Output

```
Name Length Aman 4
```

Explanation

Text length can become a useful numeric feature.

5. Binning

Code

```
df['Age_Group']=pd.cut(df['Age'], bins=[0,18,35,60], labels=['Child','Adult','Senior'])
```

Output

```
Age Age_Group 25 Adult
```

Explanation

Binning converts continuous values into categories.

6. Polynomial Features

Code

```
from sklearn.preprocessing import PolynomialFeatures  
poly=PolynomialFeatures(degree=2)
```

Output

Creates x^2 , xy and similar features.

Explanation

Useful when relationships are non-linear.

7. Comparison Table

Technique	Example	Use
Combine Columns	Total Marks	New information
Date Features	Year, Month	Time-based data
Text Features	Length	NLP basics
Binning	Age Group	Categorical ranges
Polynomial	x^2 , xy	Capture non-linear patterns

8. Common Mistakes

- Creating too many unnecessary features.
- Using future information (data leakage).
- Forgetting to engineer the same features for train and test data.

9. Interview Questions

1. What is feature engineering?
2. Why is it important?
3. Give examples of engineered features.
4. What is binning?
5. What are polynomial features?

10. Practice Questions

1. Create Total Marks.
2. Extract Year and Month from a date column.
3. Create Name Length feature.
4. Bin Age into groups.
5. Generate polynomial features.

Machine Learning Preprocessing

Chapter 6 - Feature Selection

Feature Selection means choosing only the most useful input features for training a machine learning model. It improves accuracy, reduces overfitting and speeds up training.

1. Why Feature Selection?

Code

Dataset: Age, Salary, City, Experience, Target

Output

Useful features are selected; irrelevant ones are removed.

Explanation

Removing unnecessary columns reduces noise, training time and model complexity.

2. Correlation Method

Code

```
import seaborn as sns print(df.corr(numeric_only=True))
```

Output

Correlation matrix with values between -1 and 1

Explanation

Highly correlated features may carry similar information.

3. Variance Threshold

Code

```
from sklearn.feature_selection import VarianceThreshold  
selector=VarianceThreshold(threshold=0.1) X_new=selector.fit_transform(X)
```

Output

Low-variance columns removed

Explanation

Features with almost constant values contribute very little.

4. SelectKBest

Code

```
from sklearn.feature_selection import SelectKBest, f_classif  
selector=SelectKBest(score_func=f_classif, k=3)  
X_new=selector.fit_transform(X, y)
```

Output

Top 3 features selected

Explanation

Chooses the highest-scoring features using statistical tests.

5. Recursive Feature Elimination (RFE)

Code

```
from sklearn.feature_selection import RFE from sklearn.linear_model
import LogisticRegression
rfe=RFE(LogisticRegression(),n_features_to_select=2)
X_new=rfe.fit_transform(X,y)
```

Output

Best 2 features retained

Explanation

RFE repeatedly removes the least important features.

6. Feature Importance

Code

```
from sklearn.ensemble import RandomForestClassifier
model=RandomForestClassifier() model.fit(X,y)
print(model.feature_importances_)
```

Output

[0.42 0.33 0.18 0.07]

Explanation

Tree-based models estimate the importance of each feature.

7. Comparison Table

Method	Best For	Advantage
Correlation	Numeric Data	Simple & Fast
VarianceThreshold	Low Variance Features	Removes useless columns
SelectKBest	Statistical Selection	Top K features
RFE	Model-based	High-quality selection
Feature Importance	Tree Models	Ranks all features

8. Common Mistakes

- Removing useful features without checking their importance.
- Performing feature selection before splitting the dataset (data leakage).
- Using correlation alone for every problem.
- Ignoring domain knowledge.

9. Interview Questions

1. What is feature selection?
2. Feature selection vs feature engineering?
3. What is SelectKBest?
4. Explain RFE.
5. How do tree models calculate feature importance?

10. Practice Questions

1. Find the correlation matrix of a dataset.
2. Remove low-variance features.
3. Select the best 5 features using SelectKBest.
4. Apply RFE with Logistic Regression.
5. Display feature importance using Random Forest.

Machine Learning Preprocessing

Chapter 7 - Complete Data Preprocessing Pipeline

This chapter combines all preprocessing steps into a single workflow used before training a machine learning model.

1. Pipeline Workflow

Code

Read Dataset → Explore → Missing Values → Encoding → Scaling → Feature Selection → Train-Test Split → Model Ready

Explanation

Every ML project follows these preprocessing steps before model training.

2. Load Dataset

Code

```
import pandas as pd
df=pd.read_csv('students.csv')
print(df.head())
```

Output

```
Name Gender Age Marks Aman Male 20 85 Riya Female NaN 92
```

Explanation

Load the dataset and inspect the first few records.

3. Explore Dataset

Code

```
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

Output

```
Age 1 Marks 1
```

Explanation

Check data types, summary statistics and missing values.

4. Handle Missing Values

Code

```
df['Age']=df['Age'].fillna(df['Age'].median())
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Output

Missing values replaced.

Explanation

Median is often used for outlier-prone data, mean for normally distributed numeric data.

5. Encode Categorical Data

Code

```
df=pd.get_dummies(df,columns=['Gender'],dtype=int)
```

Output

Gender_Female Gender_Male

Explanation

Convert text categories into numeric columns.

6. Feature Scaling

Code

```
from sklearn.preprocessing import StandardScaler scaler=StandardScaler()  
df[['Age', 'Marks']] = scaler.fit_transform(df[['Age', 'Marks']])
```

Output

Age & Marks scaled.

Explanation

Scale numeric features so they have similar ranges.

7. Feature Selection

Code

```
X=df.drop('Result',axis=1) y=df['Result']
```

Output

X = Features y = Target

Explanation

Separate input features from the target variable.

8. Train-Test Split

Code

```
from sklearn.model_selection import train_test_split X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Output

80% Train 20% Test

Explanation

Split the dataset before model training and evaluation.

9. Complete Pipeline Code

Code

```
Read CSV → fillna() → get_dummies() → StandardScaler() → X/y →  
train_test_split()
```

Output

Dataset ready for ML model.

Explanation

These steps form a complete preprocessing pipeline.

10. Pipeline Summary

Step	Function
Load Dataset	read_csv()

Explore	head(), info(), describe()
Missing Values	fillna()
Encoding	get_dummies()
Scaling	StandardScaler()
Feature Selection	drop()
Train-Test Split	train_test_split()

11. Common Mistakes

- Scaling before train-test split (data leakage).
- Training on raw data.
- Forgetting to save fitted encoders/scalers.
- Ignoring missing values.

12. Interview Questions

1. What is a preprocessing pipeline?
2. Why handle missing values first?
3. Why encode categorical variables?
4. What is data leakage?
5. Why split data before training?

13. Practice Questions

1. Load a CSV dataset.
2. Display info() and describe().
3. Fill missing values.
4. Encode Gender using get_dummies().
5. Scale Age and Marks.
6. Create X and y.
7. Perform an 80-20 train-test split.
8. Write the complete preprocessing pipeline.

Machine Learning Preprocessing

Chapter 8 - End-to-End Mini Machine Learning Project

This chapter combines all preprocessing concepts on a student performance dataset and prepares it for machine learning.

1. Problem Statement

Code

Predict whether a student will Pass or Fail using Age, Gender, Study Hours and Marks.

Explanation

Goal: Prepare raw data so it can be used by a machine learning algorithm.

2. Load Dataset

Code

```
import pandas as pd df=pd.read_csv('students.csv') print(df.head())
```

Output

Name Gender Age Hours Marks Result

Explanation

Load and inspect the dataset.

3. Explore Dataset

Code

```
df.info() df.describe() df.isnull().sum()
```

Output

Displays data types, statistics and missing values.

Explanation

Understand the dataset before cleaning.

4. Handle Missing Values

Code

```
df['Age']=df['Age'].fillna(df['Age'].median())  
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Output

Missing values replaced.

Explanation

Use median for Age and mean for Marks.

5. Encode Categorical Data

Code

```
df=pd.get_dummies(df,columns=['Gender'],dtype=int)
```

Output

Gender_Female Gender_Male

Explanation

Convert categorical values into numeric columns.

6. Scale Numerical Features

Code

```
from sklearn.preprocessing import StandardScaler scaler=StandardScaler()  
df[['Age', 'Hours', 'Marks']] = scaler.fit_transform(df[['Age', 'Hours', 'Marks']])
```

Output

Scaled features (mean≈0, std≈1).

Explanation

Standardization improves many ML algorithms.

7. Prepare Features and Target

Code

```
X=df.drop('Result',axis=1) y=df['Result']
```

Output

X = Features y = Target

Explanation

Separate input variables from the output label.

8. Train-Test Split

Code

```
from sklearn.model_selection import train_test_split X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Output

80% Train 20% Test

Explanation

Reserve unseen data for evaluation.

9. Dataset Ready

Code

```
print(X_train.shape) print(X_test.shape)
```

Output

Example: (80,5) (20,5)

Explanation

The processed dataset is now ready to train any ML model.

10. Complete Workflow Summary

Pipeline Step	Tool / Function
Load Data	read_csv()

Explore	head(), info(), describe()
Clean	fillna()
Encode	get_dummies()
Scale	StandardScaler()
Select X & y	drop()
Split	train_test_split()
Train Model	Any ML algorithm

11. Common Mistakes

- Training on uncleaned data.
- Performing scaling before the train-test split (data leakage).
- Forgetting to encode categorical features.
- Fitting scalers on the test set.

12. Interview Questions

1. Explain an end-to-end preprocessing pipeline.
2. Why is encoding necessary?
3. Why should scaling be fitted on training data?
4. What is data leakage?
5. Which preprocessing step comes before model training?

13. Practice Exercise

Build the complete preprocessing pipeline on another dataset (Titanic, Iris or House Prices): load the data, inspect it, handle missing values, encode categorical features, scale numerical columns, create X and y, and perform an 80–20 train-test split.